TASK 3 – Choose Variables and Name the Constants

3.1 Naming Rules

Python variable names must follow these rules:

1. A name must begin with a letter or underscore.
2. A name cannot begin with a digit.
3. Spaces are not allowed.
4. Python keywords cannot be used as variable names.
5. Variable names are case-sensitive.

Examples:

farmer_id = "F101"

litres = 10.5

fat_percentage = 4.2

snf_percentage = 8.7

3.2 Constants

BASE_RATE = 30.00

STANDARD_FAT = 3.5

STANDARD_SNF = 8.5

FAT_STEP_VALUE = 0.40

SNF_STEP_VALUE = 0.30

STEP_SIZE = 0.1

Constants are written in uppercase letters by convention.

Python does not enforce constants. A programmer can still change BASE_RATE, but uppercase naming communicates that it should remain unchanged during normal program execution.

3.3 Dynamic Typing

Python is dynamically typed. A variable can refer to values of different types at different times.

```
value = 10
print(type(value))

value = "Milk"
print(type(value))
```

Output:

```
<class 'int'>
<class 'str'>
```

Python determines the type from the value assigned to the variable.

3.4 Multiple Assignment and Swapping

```
a, b = 10, 20
a, b = b, a

print(a)
print(b)
```

Output:

```
20
10
```

Python allows multiple assignment and swapping without requiring a temporary variable.

3.5 Alternative Concept: Magic Numbers and Dictionary

Magic Numbers

```
rate = 30.00 + ((fat - 3.5) / 0.1) * 0.40
rate += ((snf - 8.5) / 0.1) * 0.30
```

The values are embedded directly in the expressions.

If the rate changes, every occurrence must be checked and updated.

Dictionary Rate Card

```
RATE_CARD = {
    "base": 30.00,
    "fat": 0.40,
    "snf": 0.30
}
```

Usage:

```
rate = RATE_CARD["base"]
```

A dictionary groups related values and can be convenient when several rate cards are required.

Comparison

Method	**Rate revision**	**Time**	**Space**
Magic numbers	Change k occurrences	O(1) per calculation	O(1)
Named constants	Change one definition per value	O(1) per calculation	O(1)
Dictionary	Change one dictionary entry per value	O(1) average lookup	O(1) for a fixed-size rate card

Conclusion: Named constants are appropriate for the original problem. A dictionary becomes more useful when the system needs multiple seasonal rate cards.

TASK 4 – Write Correct Expressions

4.1 Expressions for Payment

The fat and SNF step counts are calculated as follows:

$$\text{Fat Steps} = \text{round}\left(\frac{\text{Fat} - 3.5}{0.1}\right)$$

$$\text{SNF Steps} = \text{round}\left(\frac{\text{SNF} - 8.5}{0.1}\right)$$

The rate per litre is:

$$\text{Rate} = 30 + (\text{Fat Steps} \times 0.40) + (\text{SNF Steps} \times 0.30)$$

The payment amount is:

$$\text{Payment} = \text{Litres} \times \text{Rate}$$

The final rate and payment are rounded to two decimal places.

4.2 Python Implementation

```
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30

fat = 4.2
snf = 8.7
litres = 8.5

fat_steps = round((fat - 3.5) / 0.1)
snf_steps = round((snf - 8.5) / 0.1)

rate = round(
    BASE_RATE
    + fat_steps * FAT_STEP_VALUE
```

```
    + snf_steps * SNF_STEP_VALUE,
    2
)

amount = round(litres * rate, 2)

print("Fat steps:", fat_steps)
print("SNF steps:", snf_steps)
print("Rate:", rate)
print("Payment:", amount)
```

Output:

Fat steps: 7

SNF steps: 2

Rate: 33.4

Payment: 283.9

```
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30

fat = 4.2
snf = 8.7
litres = 8.5

fat_steps = round((fat - 3.5) / 0.1)
snf_steps = round((snf - 8.5) / 0.1)

rate = round(
    BASE_RATE
    + fat_steps * FAT_STEP_VALUE
    + snf_steps * SNF_STEP_VALUE,
    2
)

amount = round(litres * rate, 2)

print("Fat steps:", fat_steps)
print("SNF steps:", snf_steps)
print("Rate:", rate)
print("Payment:", amount)
```

```
Fat steps: 7
SNF steps: 2
Rate: 33.4
Payment: 283.9
```

4.3 Operator Precedence and Associativity

Python evaluates expressions according to operator precedence.

For example:

```
>>> 2 + 3 * 4
14

>>> (2 + 3) * 4
20

>>> 2 ** 3 ** 2
512

>>> -2 ** 2
-4

>>> (-2) ** 2
4
```

Explanation:

- Multiplication is evaluated before addition.
- Parentheses can change the evaluation order.
- Exponentiation is right-associative.
- Exponentiation has higher precedence than unary minus, so -2 ** 2 means -(2 ** 2).

4.4 Floating-Point Surprises

Surprise 1

```
>>> (4.2 - 3.5) / 0.1
7.000000000000001
```

The result differs slightly from 7 because of floating-point representation.

Use:

>>> round((4.2 - 3.5) / 0.1)

7

Surprise 2

>>> 0.1 + 0.2 == 0.3

False

The binary floating-point values are not represented exactly.

Surprise 3

>>> round(2.5)

2

>>> round(3.5)

4

Python uses round-half-to-even for exact halfway cases. Therefore, round(2.5) produces 2, not 3.

4.5 Difference Between round(), int() and math.floor()

import math

print(round(-2.7))

print(int(-2.7))

print(math.floor(-2.7))

Output:

-3

-2

-3

- round() rounds to the nearest integer, with ties handled using round-half-to-even.
- int() truncates the fractional part towards zero.

- math.floor() returns the greatest integer less than or equal to the number.

4.6 Comparison Chaining and Boolean Operators

fat = 4.2

print(2.0 <= fat <= 7.0)

Output:

True

This is equivalent to:

2.0 <= fat and fat <= 7.0

Boolean operators include and, or and not.

Example:

if 2.0 <= fat <= 7.0 and snf >= 0:

    print("Valid")

4.7 Alternative Concept: int() or math.floor() for Steps

For fat = 4.2:

$$\frac{4.2 - 3.5}{0.1} \approx 7$$

For fat = 3.1:

$$\frac{3.1 - 3.5}{0.1} \approx -4$$

The following code compares the methods:

import math

for fat in (4.2, 3.1):

    steps = (fat - 3.5) / 0.1

    print(

```
        fat,
        "round:", round(steps),
        "int:", int(steps),
        "floor:", math.floor(steps)
    )
```

Expected step counts:

Fat	round()	int()	math.floor()
4.2	7	7	7
3.1	-4	-3	-4

The precise raw floating-point values should be checked on the machine used for the assignment. The important distinction is that int() truncates negative numbers towards zero, whereas floor() rounds down.

All three operations have **O(1) time and O(1) additional space** for one value.

Conclusion: round() is the appropriate choice for the specified step-count rule, but the final design challenge requires more careful handling of two-decimal analyser readings.